

CONSTRAINT-BASED EXAM SCHEDULING

a) PEAS Description

PEAS stands for Performance Measure, Environment, Actuators, and Sensors. For this exam-scheduling agent, the PEAS description is given below.

PEAS Component	Description
Performance Measure	The agent should assign all five courses to one of the three exam slots. Every pair of courses with shared students must be scheduled in different slots. The timetable should be complete and conflict-free.
Environment	The environment consists of five courses: Artificial Intelligence (A), Database Systems (B), Computer Networks (C), Operating Systems (D), and Software Engineering (E), along with three available exam slots and the student-enrollment conflicts.
Actuators	The agent assigns courses to slots, changes or undoes assignments during search, and presents the final timetable.
Sensors	The agent receives the course list, available slots, conflict-pair information, and the current partial timetable. It checks the legal slots available for each unassigned course.

The main objective of the agent is to produce a complete and conflict-free examination timetable.

b) Suitable Intelligent Agent

A goal-based agent is the most suitable intelligent agent for this problem. Its goal is to assign every course to an exam slot such that all conflicting pairs of courses are scheduled in different slots.

The agent explores possible assignments, checks whether they satisfy the constraints, and uses backtracking if a choice prevents it from completing a valid timetable.

A simple reflex agent is less suitable because it makes decisions based only on the current input and may not consider the effects on the remaining timetable. A model-based agent can maintain information about the partial schedule and constraints, but the goal-based agent best represents the search for a solution that satisfies the required goal.

A utility-based agent could also be used if additional preferences were introduced, such as spreading examinations across different days or minimizing room usage. Since the problem only requires a conflict-free timetable, a goal-based agent is sufficient.

c) CSP Formulation and Constraint Graph

A Constraint Satisfaction Problem (CSP) consists of three main components: variables, domains, and constraints.

1. Variables

Each course is represented by a variable.

$$X = \{A, B, C, D, E\}$$

Where:

A – Artificial Intelligence

B – Database Systems

C – Computer Networks

D – Operating Systems

E – Software Engineering

2. Domains

Each course can be assigned to any one of the three available exam slots.

$$D(A) = D(B) = D(C) = D(D) = D(E)$$

$$= \{\text{Slot1}, \text{Slot2}, \text{Slot3}\}$$

3. Constraints

The following pairs of courses have shared students. Therefore, each pair must be scheduled in different exam slots.

$$A \neq B$$

$$A \neq C$$

$$B \neq C$$

$$B \neq D$$

$$C \neq D$$

$$C \neq E$$

$$D \neq E$$

For example, $A \neq B$ means that Artificial Intelligence and Database Systems cannot be scheduled in the same exam slot.

4. Constraint Graph

A constraint graph represents each course as a vertex and each conflict between two courses as an edge.

$$\text{Vertices} = \{A, B, C, D, E\}$$

$$\text{Edges} = \{(A, B), (A, C), (B, C), (B, D), (C, D), (C, E), (D, E)\}$$

Every edge indicates that the two connected courses must be assigned different exam slots.

d) Plain Backtracking Search

Plain backtracking search assigns one variable at a time in the following order:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

For each course, the algorithm tries the values in the order Slot1, Slot2, and Slot3. An assignment is rejected if it conflicts with a previously assigned course.

If all values for a variable fail, the algorithm backtracks to the previous variable and tries another available value.

Steps of Plain Backtracking Search

Step	Assignment Attempt	Decision and Explanation
1	A = Slot1	Accepted. A has no previously assigned course to conflict with.
2	B = Slot1	Rejected. B conflicts with A, and both would be in Slot1.
3	B = Slot2	Accepted. A and B are assigned to different slots.
4	C = Slot1	Rejected. C conflicts with A, which is in Slot1.
5	C = Slot2	Rejected. C conflicts with B, which is in Slot2.
6	C = Slot3	Accepted. C is different from both A and B.
7	D = Slot1	Accepted. D conflicts with B and C, which are in Slot2 and Slot3.
8	E = Slot1	Rejected. E conflicts with D, which is in Slot1.
9	E = Slot2	Accepted. E differs from both C and D.

Final Timetable

Course	Exam Slot
Artificial Intelligence (A)	Slot1
Database Systems (B)	Slot2
Computer Networks (C)	Slot3
Operating Systems (D)	Slot1
Software Engineering (E)	Slot2

The final timetable satisfies all the given conflict constraints.

There were 9 assignment attempts, including 4 rejected attempts and 0 backtracks. No previously accepted assignment needed to be undone.

e) Backtracking with Forward Checking

Forward checking is an improvement over plain backtracking. After assigning a value to a variable, it immediately removes that value from the domains of unassigned variables that conflict with it.

This helps identify forced choices or empty domains before the search reaches those variables.

For simplicity, Slot1, Slot2, and Slot3 are represented as 1, 2, and 3.

Steps of Forward Checking

Step	Assignment or Attempt	Remaining Domains of Unassigned Variables
1	Assign A = 1	B = {2, 3}, C = {2, 3}, D = {1, 2, 3}, E = {1, 2, 3}. Slot1 is removed from B and C because they conflict with A.
2	Try B = 1	Rejected because A and B conflict. The domains remain unchanged.
3	Assign B = 2	C = {3}, D = {1, 3}, E = {1, 2, 3}. Slot2 is removed from C and D because they conflict with B.
4	Try C = 1	Rejected because C conflicts with A.
5	Try C = 2	Rejected because C conflicts with B.
6	Assign C = 3	D = {1}, E = {1, 2}. Slot3 is removed from D and E because they conflict with C.
7	Assign D = 1	E = {2}. Slot1 is removed from E because D and E conflict.
8	Assign E = 2	Accepted. All courses are assigned and all constraints are satisfied.

Final Timetable

Course	Exam Slot
Artificial Intelligence (A)	Slot1
Database Systems (B)	Slot2
Computer Networks (C)	Slot3
Operating Systems (D)	Slot1
Software Engineering (E)	Slot2

Forward checking reduces the domains of unassigned variables after every successful assignment. After assigning $C = \text{Slot3}$, D has only Slot1 available. Once $D = \text{Slot1}$ is assigned, E has only Slot2 available.

Thus, forward checking makes the remaining choices easier to identify.

f) Comparison Between Plain Backtracking and Forward Checking

Both methods use the same variable order and value order. However, forward checking removes conflicting values from the domains of unassigned variables immediately after each assignment.

Measure	Plain Backtracking	Backtracking with Forward Checking
Assignment attempts	9	9
Rejected attempts	4	4
Backtracks	0	0
Final timetable found	Yes	Yes
Domain reduction	Checks conflicts when assigning a variable	Removes conflicting values immediately
Early detection of failure	Limited	Better

In this example, both methods require 9 assignment attempts and find the same valid timetable without undoing any accepted assignment.

However, forward checking reduces the domains of unassigned variables earlier. For example, after assigning $C = \text{Slot3}$, it determines that D must be assigned Slot1. After assigning $D = \text{Slot1}$, E must be assigned Slot2.

In a more difficult problem, an assignment may leave a future variable with no legal values. Plain backtracking may discover this only when it reaches that variable, whereas forward checking can detect the empty domain immediately after the assignment that caused it.

Therefore, forward checking can reduce unnecessary search and improve efficiency, although it does not reduce the number of assignment attempts in this particular example.

CONCLUSION

Constraint-Based Exam Scheduling can be modelled as a Constraint Satisfaction Problem using variables, domains, and constraints. A goal-based agent can solve the problem by searching for a complete timetable that satisfies all constraints.

Both plain backtracking and backtracking with forward checking produce a valid timetable. However, forward checking is generally more efficient because it reduces the domains of unassigned variables and can detect failures earlier.